

Université Abdelmalek Essaâdi
Faculté des Sciences de Tétouan
Département Informatique

Licence IIDL / S5 :
Ingénierie Informatique et Développement Logiciel

Développement d'applications en Java

Pr. Otman ABDOUN
Département informatique, Faculté des sciences
Université Abdelmalek Essaâdi, Tétouan

Année universitaire 2026–2027

LICENCE IIDL— SEMESTRE 5

Développement d'applications en Java

Élément 1 — Fondamentaux et Programmation Orientée Objet en Java

CH1

Fondamentaux de Java

CH2

Programmation orientée objet

CH3

API Java et programmation avancée

Année universitaire 2026–2027

Plan de l'Élément 1

CH1	**Fondamentaux de Java**	*JVM, syntaxe, structures de base*
CH2	**Programmation orientée objet en Java**	*Classes, héritage, polymorphisme*
CH3	**API Java et programmation avancée**	*Exceptions, collections, flux*

INTRODUCTION

Objectifs de l'Élément 1

1. **Maîtriser la syntaxe Java**
   Comprendre la JVM, écrire, compiler et exécuter un programme Java structuré avec Maven.

2. **Programmer en objet**
   Concevoir des classes, utiliser héritage, polymorphisme, classes abstraites et interfaces.

3. **Exploiter les API standards**
   Gérer exceptions, chaînes, collections, dates, fichiers et flux de données (Streams).

4. **Préparer la suite du module**
   Poser les bases nécessaires à JDBC, Swing et à l'architecture MVC/DAO (Éléments 2 et 3).

CHAPITRE 1

Fondamentaux de Java

- Historique et caractéristiques du langage
- Installation et environnement de développement
- Variables, types, opérateurs, structures de contrôle
- JDK, JVM, JRE et exécution d'un programme
- Organisation d'un projet Java/Maven
- Tableaux, méthodes et paramètres

1. HISTORIQUE

Historique et caractéristiques de Java

1

Origines (1995, Sun Microsystems)

Créé par James Gosling pour des systèmes embarqués ("Oak"), puis adapté au Web sous le nom Java. Racheté par Oracle en 2010.

2

"Write Once, Run Anywhere" (WORA)

Le code source est compilé en bytecode indépendant de la plateforme, exécuté par une JVM spécifique à chaque OS.

3

Langage orienté objet

Tout (ou presque) est objet ; syntaxe proche du C/C++ mais sans pointeurs explicites ni héritage multiple de classes.

4

Robustesse et portabilité

Typage fort, gestion automatique de la mémoire (Garbage Collector), gestion structurée des exceptions.

5

Écosystème riche

API standard très large, Maven/Gradle pour la gestion de projet, immense communauté et bibliothèques (Spring, Hibernate...).

6

Versions LTS

Java 8, 11, 17, 21 (**LTS**). Ce module utilise Java 27, la version recommandée en production en 2026.

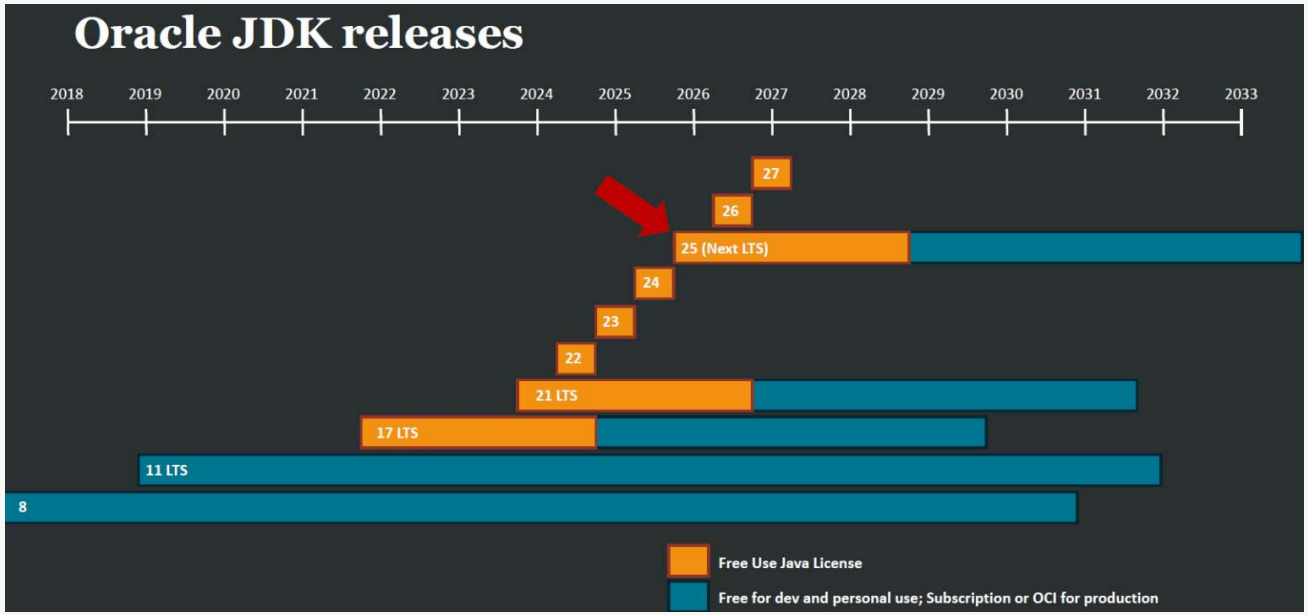
Historique de Java

1991	Début du projet "Green" chez Sun Microsystems par James Gosling
1995	Première version publique de Java (Java 1.0)
1996	Introduction de Java Applets pour le web
1998	Java 2 (J2SE 1.2) - Refonte majeure de la plateforme
2004	Java 5 - Ajout des génériques, annotations, autoboxing
2010	Oracle acquiert Sun Microsystems
2014	Java 8 - Expressions lambda, Stream API
2018	Nouveau cycle de release tous les 6 mois
2023	Java 21
2025	Java 25 (LTS)
2026	Java 27 now available, **September 16, 2026**

Logo de Sun Microsystems, créateur de Java



James Gosling, père fondateur de Java



Caractéristiques principales de Java

Orienté objet

Tout en Java est un objet, facilitant la modularité et la réutilisation du code

Portabilité (WORA)

Write Once, Run Anywhere - Le bytecode Java s'exécute sur n'importe quelle JVM

Garbage Collector

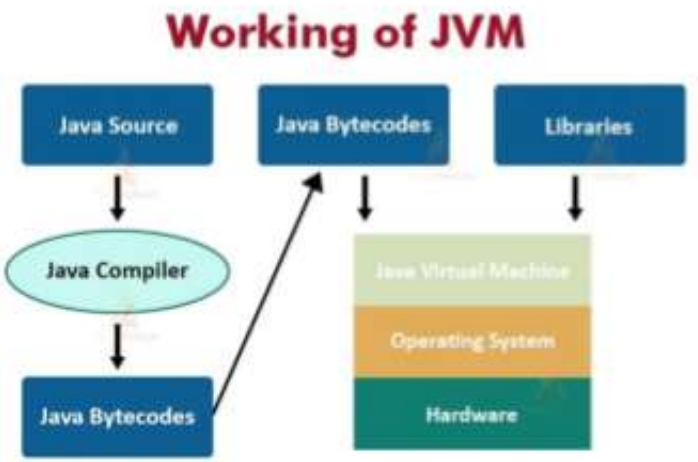
Gestion automatique de la mémoire, libérant le développeur de cette tâche

Sécurité intégrée

Architecture de sécurité robuste avec sandbox d'exécution et vérification du bytecode

Bibliothèque standard riche

API complète couvrant réseau, interface graphique, collections, concurrence, etc.



Architecture de la JVM (Java Virtual Machine) permettant la portabilité du code Java

Caractéristiques principales de Java







ORACLE

Java SE

Java mobile

J2ME platform

JavaFx

JAVA SWING

Java EE™

Caractéristiques principales de Java: Java SE

- ✓ Le terme « **Java** » fait bien évidemment référence à un langage.
- ✓ Mais également à une plate-forme : son nom complet est « **Java SE** » pour **Java Standard Edition**, et était anciennement raccourci « **J2SE** ».
- ✓ **Java SE** est constituée de nombreuses bibliothèques, ou **API**.
- ✓ Par exemple : *java.lang, java.io, java.math, java.util*, etc.
- ✓ **Java SE** regroupe toutes les bibliothèques qui contiennent un nombre important de classes et de méthodes prêtes à l'emploi pour effectuer toutes sortes de tâches et afin de développer les applications Desktop.



Caractéristiques principales de Java: Java EE (~~J2EE~~ ou ~~JEE~~)

- ✓ **Java EE** signifie **Java Enterprise Edition**.
- ✓ **Java EE** est à une extension de la plate-forme standard.
- ✓ La plate-forme **Java EE** est construite sur le langage Java et la plate-forme Java SE.
- ✓ L'objectif majeur de **Java EE** est de faciliter le développement d'applications web robustes et distribuées, déployées et exécutées sur un **serveur d'applications**.
- ✓ **Java EE 8** était la dernière version de **Java EE** d'Oracle, et maintenant, Oracle a déplacé **Java EE** vers une fondation open source (*Eclipse Foundation*), et elle s'appelle **Jakarta EE**.





1. HISTORIQUE

Java face à C / C++

C / C++ (déjà connu)

- Compilation directe en code machine natif
- Gestion manuelle de la mémoire (malloc/free, new/delete)
- Pointeurs explicites et arithmétique de pointeurs
- Héritage multiple de classes possible
- Portabilité limitée : à recompiler par plateforme
- Pas de gestion structurée d'exceptions en C

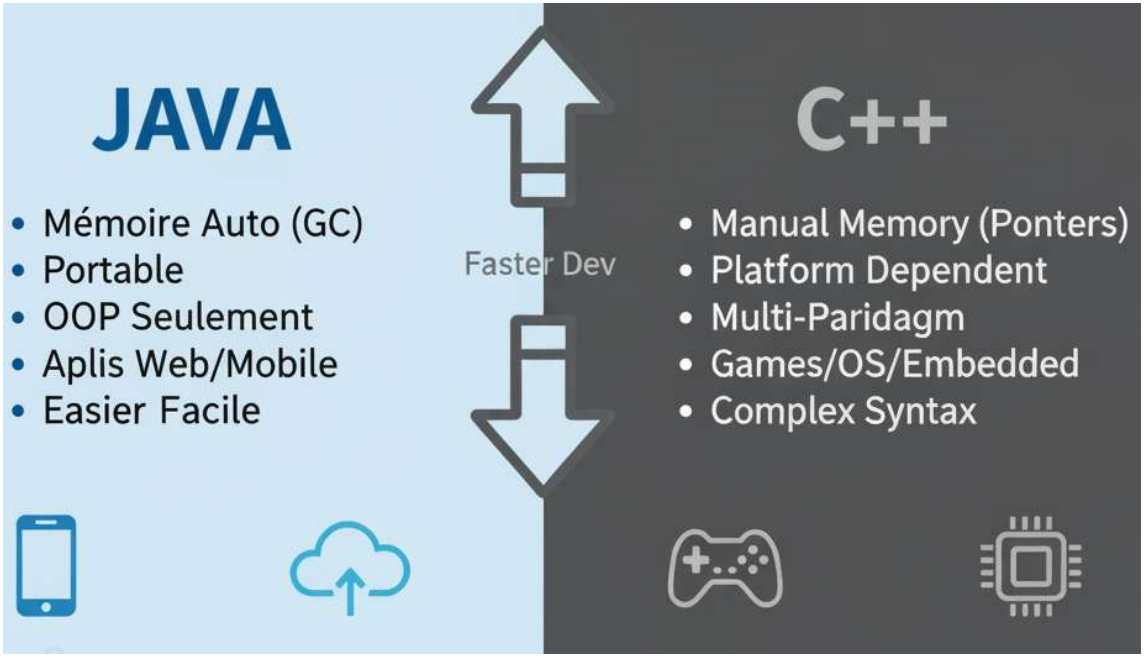
Java (nouveauté du module)

- Compilation en bytecode + exécution par la JVM
- Gestion automatique de la mémoire (Garbage Collector)
- Pas de pointeurs explicites : références sécurisées
- Héritage simple de classes + interfaces multiples
- Portabilité totale : "compilé une fois, exécuté partout"
- Gestion des exceptions obligatoire pour les erreurs vérifiées

Java face à C / C++

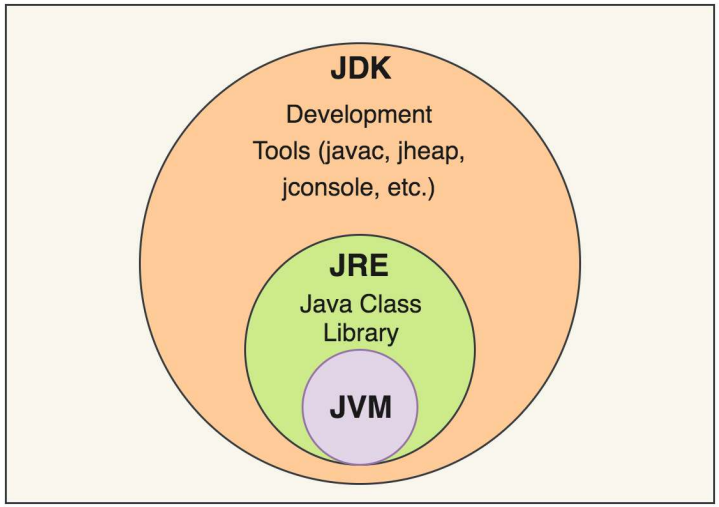
Caractéristique	Java	C/C++
Gestion mémoire	Automatique (Garbage Collector)	Manuelle (malloc/free, new/delete)
Pointeurs	Références uniquement	Pointeurs explicites
Héritage	Simple uniquement	Simple et multiple
Portabilité	Élevée (WORA)	Dépendante de la plateforme
Compilation	Bytecode → JVM	Code machine natif





JDK, JRE et JVM : bien distinguer

- ✓ JDK (Java Development Kit) : Fournit outils pour compiler, déboguer, exécuter le code Java.
- ✓ JRE (Java Runtime Environment) : Environnement nécessaire pour exécuter les applications Java.
- ✓ JVM (Java Virtual Machine) : Simule l'exécution des programmes Java.

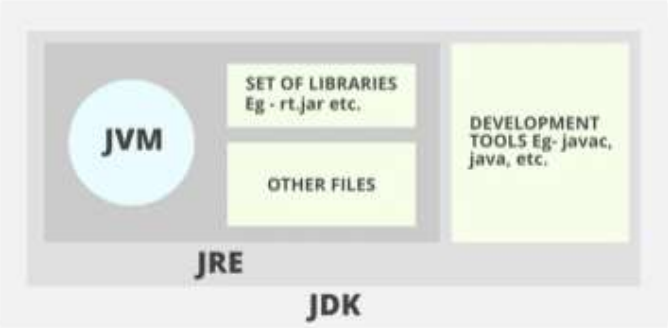


JDK, JRE et JVM : bien distinguer

JDK (Java Development Kit)

Environnement complet pour le développement d'applications Java

- Compilateur Java (javac)
- Outils de développement (javadoc, jar, etc.)
- Débogueur (jdb)
- Inclut le JRE



Relation entre JDK, JRE et JVM - Structure en couches

JRE (Java Runtime Environment)

Environnement d'exécution pour les applications Java

- Bibliothèques de classes Java (API)
- Chargeur de classes
- Inclut la JVM

JVM (Java Virtual Machine)

Machine virtuelle qui exécute le bytecode Java

- Interprète le bytecode
- Compilateur JIT (Just-In-Time)
- Garbage Collector
- Gestionnaire de mémoire

L'environnement de développement Java (IDE)

Ligne de commande

javac

Compilateur Java qui transforme le code source (.java) en bytecode (.class)

java

Lanceur de la JVM qui exécute le bytecode Java

jar

Outil pour créer et gérer des archives JAR (Java ARchive)

jdb

Débogueur Java en ligne de commande

Outils de build

Maven

Gestion de dépendances et automatisation de build

Gradle

Système de build flexible basé sur Groovy/Kotlin

Environnements de Développement Intégrés (IDE)

Eclipse IDE

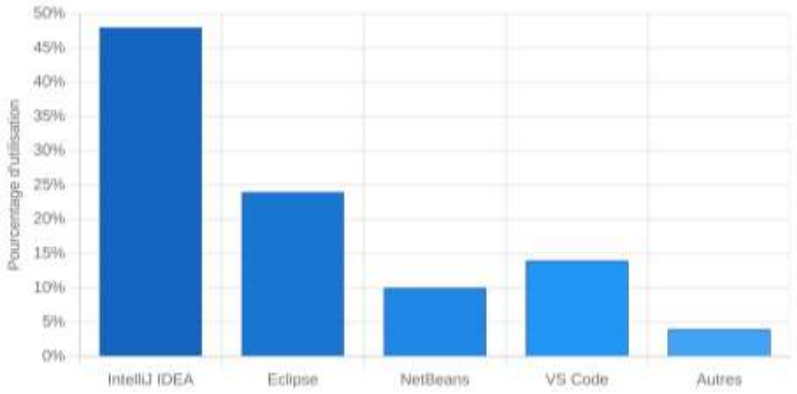
Open source, extensible via plugins

Apache NetBeans

Modulaire, multi-langage

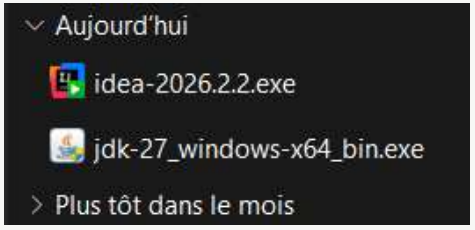
Intellij IDEA

Commercial, très puissant



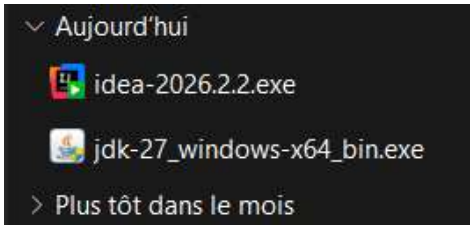
L'environnement de développement Java (IDE)

Composant	Contient	Rôle
JDK	JRE + compilateur (javac) + outils (javadoc, jar, jdb…)	Nécessaire pour développer et compiler du Java
JRE	JVM + bibliothèques standard (java.lang, java.util…)	Nécessaire pour exécuter une application Java
JVM	Interpréteur bytecode + JIT + Garbage Collector	Exécute le bytecode .class de façon portable



Pour ce module : JDK 27 LTS installé + IntelliJ IDEA comme IDE 2026,2,2 (alternatives possibles : Eclipse, NetBeans).

L'environnement de développement Java (IDE)





L'environnement de développement Java (IDE)

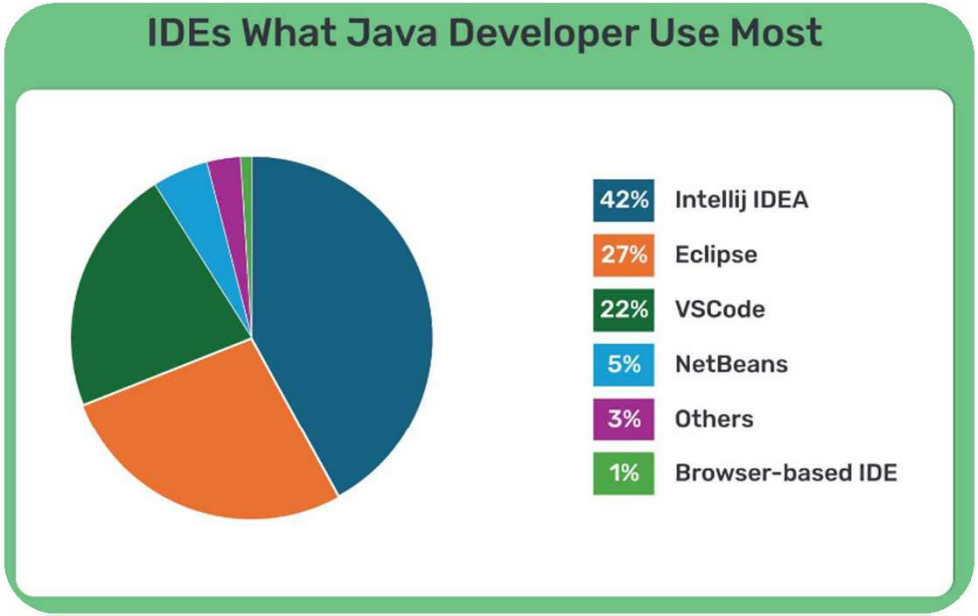
- ✓ **IDE** - Integrated Development Environment - Environnement de développement intégré; un IDE n'est pas un éditeur de texte.



- ✓ **IDE** a un ensemble de fonctionnalités :
  - La vérification de la syntaxe du langage ;
  - La détection des erreurs ;
  - L'auto-complétion ;
  - Un compilateur ;
  - Un serveur.
- ✓ Principaux **IDEs** : Eclipse, IntelliJ IDEA, NetBeans.

L'environnement de développement Java (IDE)

Apache NetBeans IDE



eclipse

Visual Studio Code

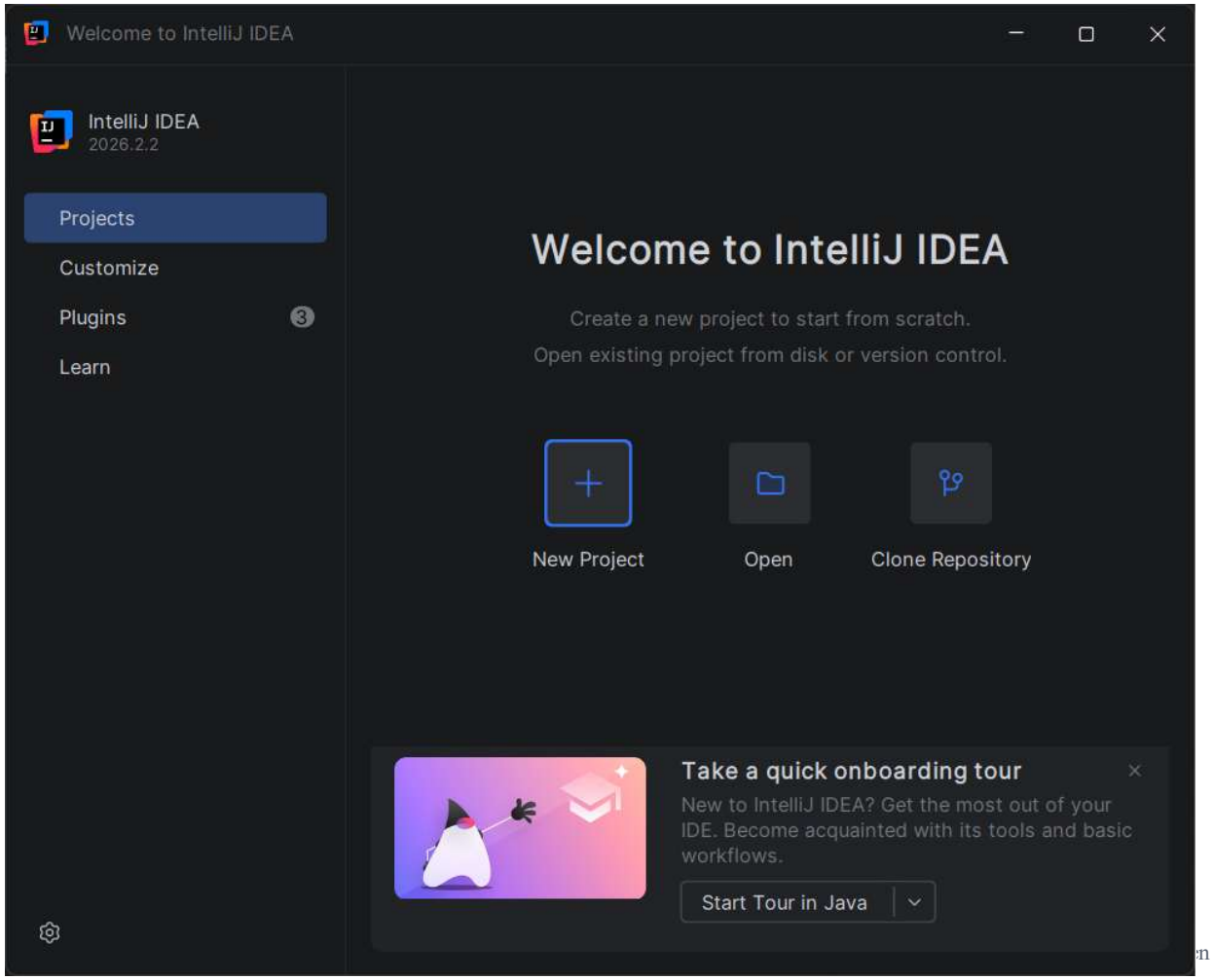
L'environnement de développement Java (IDE)





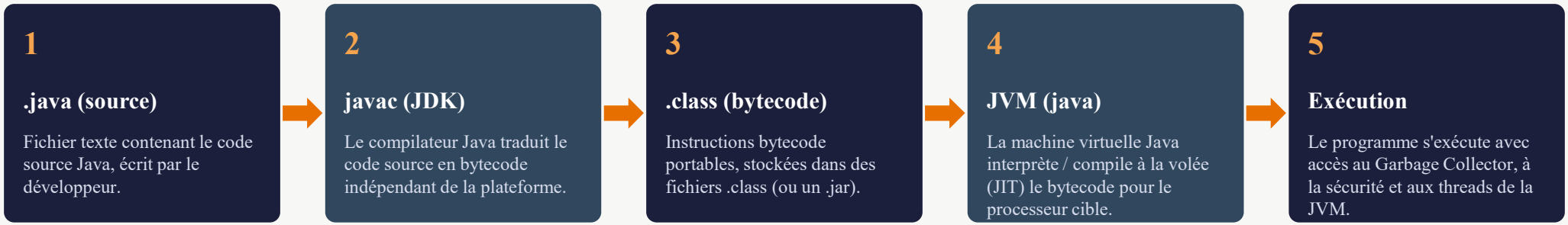
L'environnement de développement Java (IDE)





2. JDK / JVM

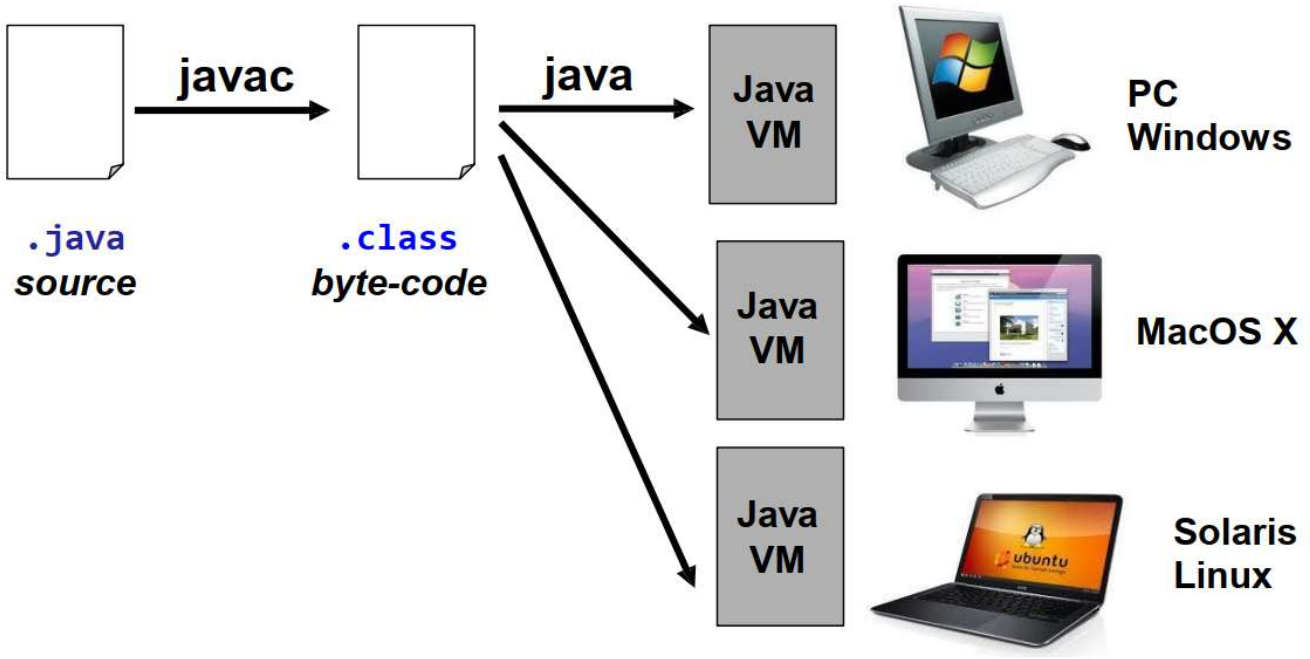
Du code source à l'exécution

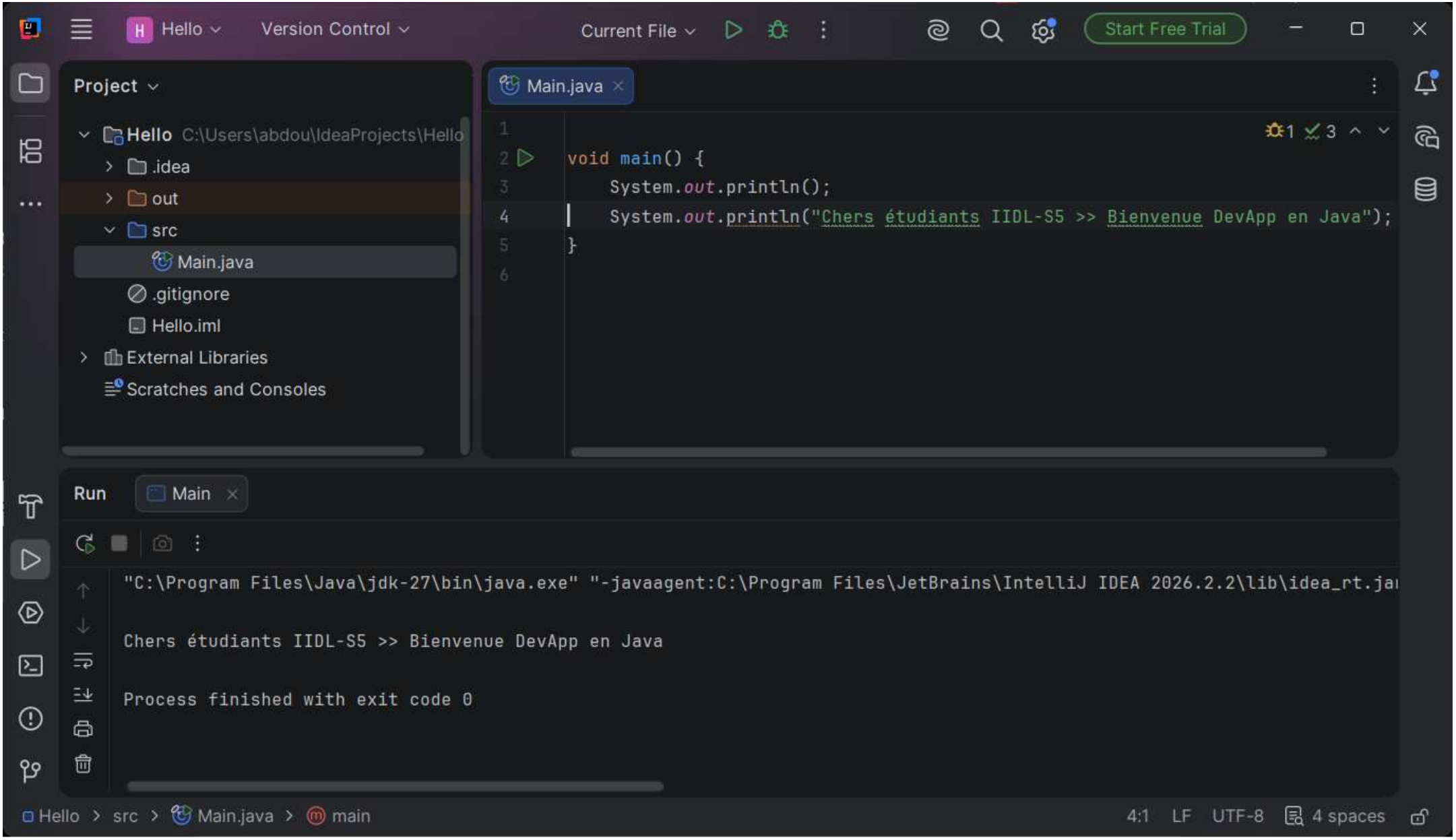


JDK (Java Development Kit) = JRE (environnement d'exécution) + outils de développement (javac, javadoc, jar…). La JVM (Java Virtual Machine) est le composant qui exécute le bytecode : elle rend Java portable sur Windows, Linux, macOS.

Du code source à l'exécution

- **byte-code assure la portabilité des programmes Java**
  - langage d'une Machine Virtuelle
  - à l'exécution un interpréteur simule cette machine virtuelle




Hello
Version Control
Current File
Start Free Trial
Project
Hello C:\Users\abdou\IdeaProjects\Hello
.idea
out
src
Main.java
.gitignore
Hello.iml
External Libraries
Scratches and Consoles
Main.java
1 3
void main() {
System.out.println();
System.out.println("Chers étudiants IIDL-S5 >> Bienvenue DevApp en Java");
}
Run
Main
"C:\Program Files\Java\jdk-27\bin\java.exe" "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA 2026.2.2\lib\idea_rt.ja
Chers étudiants IIDL-S5 >> Bienvenue DevApp en Java
Process finished with exit code 0
Hello > src > Main.java > main
4:1 LF UTF-8 4 spaces